



INDIAN INSTITUTE OF ENGINEERING SCIENCE AND TECHNOLOGY, SHIBPUR

Heart Rate Monitoring System Using ESP32 and MAX30100 Pulse Oximeter

Project by:

SANKHADEEP SARKAR (2023ITB001)

ANKITA SINGH (2023ITB004)

SUBH RAJ SHAW (2023ITB006)

RISHAV KUMAR (2023ITB007)

Acknowledgement

We would like to express our sincere gratitude to Dr. Chandan Giri, our respected professor, for providing us with the opportunity to work on this project as part of our coursework. His guidance and supervision throughout the course have been valuable in helping us gain knowledge and confidence in this subject. We are also thankful to our institution and department for providing the necessary resources and academic environment that enabled us to complete this project successfully. Finally, we express our appreciation to everyone who directly or indirectly inspired and supported us during the preparation of this work.

SANKHADEEP SARKAR

ANKITA SINGH

SUBH RAJ SHAW

RISHAV KUMAR

INDEX

1. Introduction

1.1 ESP32

1.2 Common Components of ESP32 Boards

2. Components Description

2.1 ESP32

2.2 Breadboard

2.3 Push Button

2.4 Adafruit Monochrome OLED display

2.5 MAX30100 Pulse Oximeter

2.6 Arduino Software

3. Project Description

3.1 Objective

3.2 Necessary Components

3.3 Experimental Planning

3.4 Circuit Setup

3.5 Arduino Code

3.6 Observation

3.7 Discussion

4. Conclusion

5. References

6. Arduino Code

1. Introduction

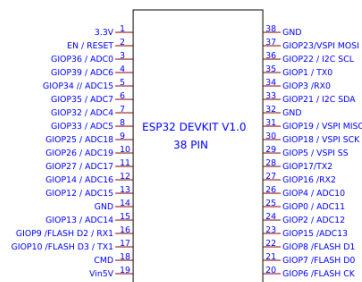
1.1 ESP32

An ESP32 is a low-cost, low-power Microcontroller with integrated WiFi and Bluetooth. It is easily programmable, uses the Arduino IDE for development, and can be used with common Arduino libraries.

1.1.2 Why use ESP32?

ESP32 provides a easy **interface for hardware connections**, largely due to the design of its development boards and its versatile General Purpose Input/Output (GPIO) pins.

1.2 Common Components of ESP32 Borards



- Power pins - 3.3V output on an ESP32 development board is generated by AMS1117
- Digital pins - GPIO 18, 19, 21, 22, 23, 25, 26, 27, 32, 33.
- Serial Data and Clock
- Enable / Reset

2. Components Description

2.1 ESP32

The ESP32 is a low-cost, low-power microcontroller with integrated Wi-Fi and Bluetooth, developed by Espressif Systems. It features a dual-core processor, ample RAM, and a variety of peripherals, making it highly versatile for a wide range of IoT and embedded applications.

2.2 Breadboard

A breadboard is a construction base for prototyping of electronics. Because the solderless breadboard does not require soldering, it is reusable. This makes it easy to use for creating temporary prototypes and experimenting with circuit design.

2.3 Push Button

A push button switch is a mechanical device used to control an electrical circuit in which the operator manually presses a button to actuate an internal switching mechanism.

2.4 Adafruit Monochrome 0.91" OLED Display

Adafruit Monochrome 0.91" 128x32 I²C OLED Display is small (only about 1" diagonal) but very readable due to the high contrast of the OLED display. This display comprises 128x32 individual white OLED pixels, each turned on or off by the controller chip.

2.5 MAX30100 Pulse Oximeter

The MAX30100 is an integrated pulse oximetry and heart rate monitor sensor solution. It combines two LEDs, a photodetector, optimized optics, and low-noise analog signal processing to detect pulse oximetry and heart-rate signals.

2.6 Arduino Software

The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. The environment is written in Java and based on Processing and other open-source software. This software can be used with any Arduino board.



a)ESP32



b)MAX30100



c)OLED Display



d)Breadboard



e) Push button

3. Project Description

3.1 Objective

The objective of this project is to get the ESP32 to take input from the MAX30100 and display the readings on the Adafruit Monochrome OLED display.

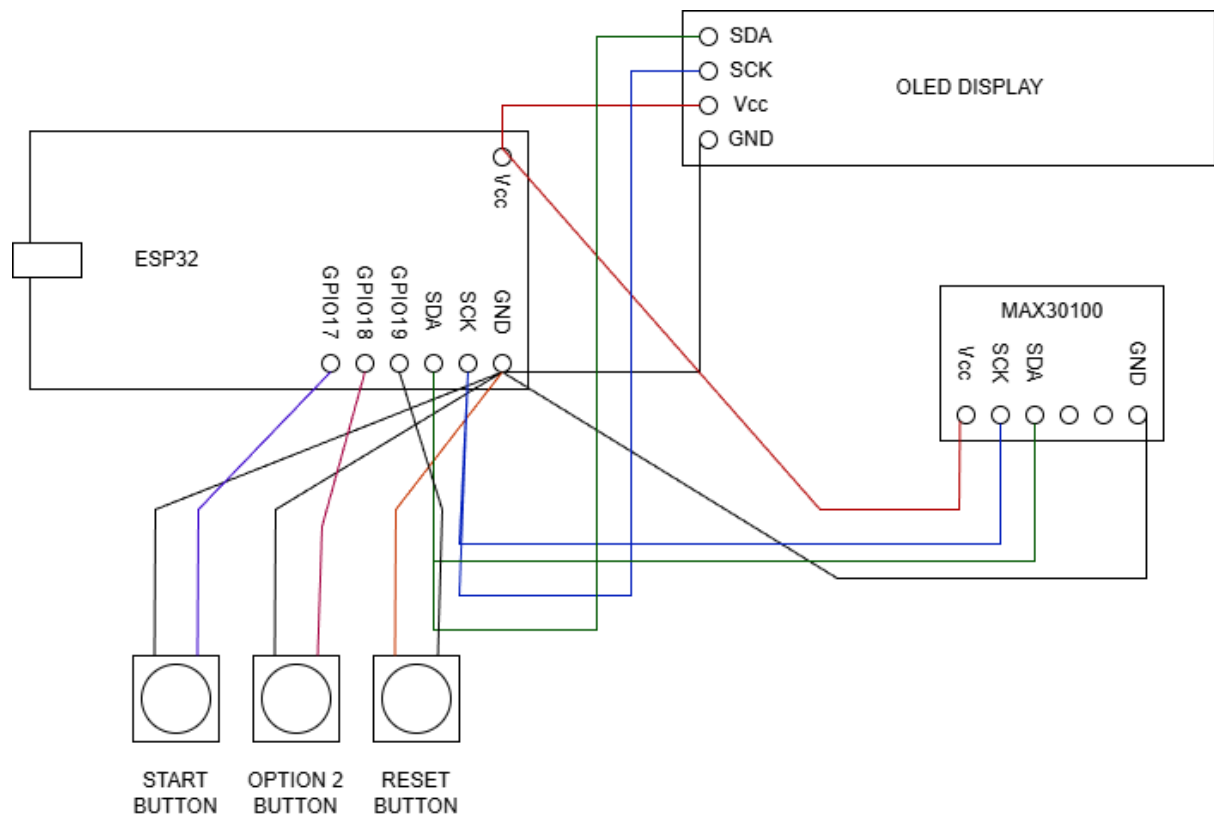
3.2 Necessary Components

ESP32, Breadboard, type-c cable, 3 Push buttons, Jumper wires, Adafruit Monochrome OLED display, MAX30100 Pulse Oximeter, Electrical Tape.

3.3 Experimental Planning

The MAX30100 Pulse Oximeter provides raw data (ie, the raw red and infrared light absorption data) in a FIFO buffer, and the microcontroller is responsible for processing this data to calculate the heart rate and blood oxygen saturation (SpO₂). The 0.91" OLED display primarily takes input from the ESP32 via the I²C communication protocol. It is a serial interface that takes only two wires for communication.

3.4 Circuit Setup

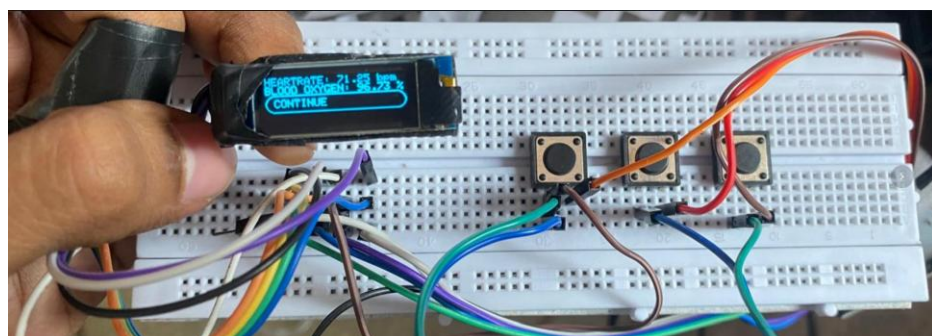


3.5 Arduino code

At the end of the report.

3.6 Observation

For a certain time duration, the ESP32 takes readings and displays the final average on the OLED display.



3.7 Discussion

At the center of the module is the MAX30100—a small but powerful sensor created by Analog Devices. This sensor is designed to measure both heart rate and blood oxygen levels (SpO₂).

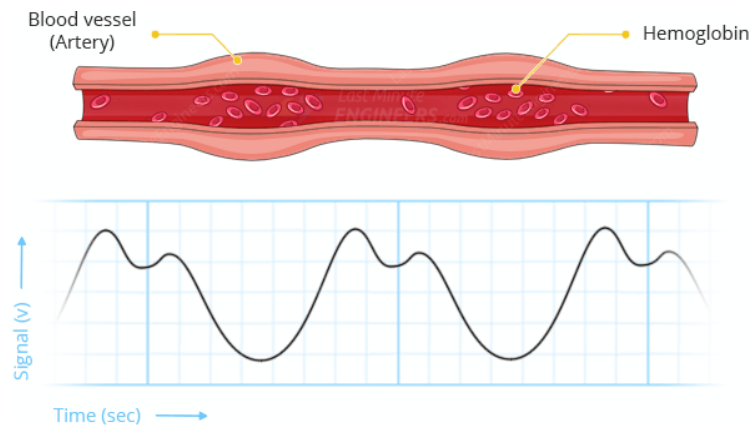
The sensor includes:

- Two special LEDs — one red and one infrared (IR) that shine light into your skin.
- A photodetector — that measures how much light bounces back.
- Special optics and signal processing — that make sure the measurements are accurate.

The sensor has a special memory storage called a FIFO buffer (which stands for “First In, First Out”). This buffer works like a temporary storage space. It allows the sensor to keep collecting heart rate and blood oxygen level data even when the microcontroller is busy.

Your blood carries oxygen with the help of a protein called hemoglobin. When hemoglobin is carrying oxygen (known as oxygenated hemoglobin, or HbO₂), it absorbs more infrared light.

Every time your heart beats, fresh oxygen-rich blood flows into your finger. Since this blood is full of oxygenated hemoglobin, it absorbs more infrared light. This means less infrared light reflects back to the photodetector.

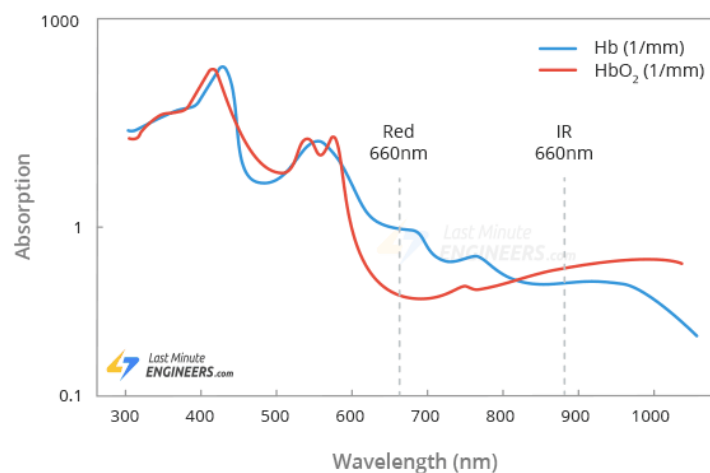


By continuously shining the infrared light and measuring these changes in the amount of light detected, the sensor can see a pattern that matches your heartbeat.

To measure the oxygen level in your blood, the MAX30100 cleverly uses both the red and infrared lights. The key idea here is that oxygenated hemoglobin (HbO_2) and deoxygenated hemoglobin (Hb) absorb light differently:

- Oxygenated hemoglobin absorbs more infrared light (880nm)
- Deoxygenated hemoglobin absorbs more red light (660nm)

The absorption-spectrum graph below shows how oxygenated hemoglobin and deoxygenated hemoglobin absorb different wavelengths of light at different rates.



By comparing how much red light and how much infrared light is absorbed, the MAX30100 can calculate what percentage of your hemoglobin is carrying oxygen. This percentage is your SpO_2 level, which indicates how well-oxygenated your blood is.

4. Conclusion

According to all the observation discussion, we can conclude that the MAX30100 accurately measures the heart rate and blood oxygen level.

5. References

1. [Arduino-MAX30100/examples/MAX30100_Minimal/MAX30100_Minimal.ino at master · oxullo/Arduino-MAX30100 · GitHub](#)
2. [Adafruit_SSD1306/examples/ssd1306_128x32_i2c/ssd1306_128x32_i2c.ino at master · adafruit/Adafruit_SSD1306 · GitHub](#)
3. [Adafruit-GFX-Library/.github at master · adafruit/Adafruit-GFX-Library · GitHub](#)
4. [\(PDF\) Arduino Project Report - Academic Project](#)
5. [Interfacing MAX30100 Pulse Oximeter and Heart Rate Sensor with Arduino](#)

6. Arduino Code:

```
#include <Wire.h>

#include "MAX30100_PulseOximeter.h"

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>


#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 32

#define SCREEN_ADDRESS 0x3C

#define SCREEN_RESET -1


#define REPORTING_PERIOD_MS 999


PulseOximeter sensor;


Adafruit_SSD1306 screen(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
SCREEN_RESET);


#define START 17

#define PREV 18

#define RESET 19


uint32_t tsLastReport = 0;


int count = 0;
```

```
unsigned long startTime;
```

```
unsigned long endTime;
```

```
unsigned long duration;
```

```
void onBeatDetected() {}
```

```
void displayMenu() {
```

```
    while(true) {
```

```
        screen.clearDisplay();
```

```
        screen.drawRoundRect(0, 0, screen.width(), screen.height()/2,
```

```
            screen.height()/4, SSD1306_WHITE);
```

```
        screen.setTextSize(1);
```

```
        screen.setTextColor(SSD1306_WHITE);
```

```
        screen.setCursor(screen.height()/4, screen.height()/8);
```

```
        screen.println("START");
```

```
        screen.drawRoundRect(0, screen.height()/2, screen.width(),  
screen.height()/2,
```

```
            screen.height()/4, SSD1306_WHITE);
```

```
        screen.setTextSize(1);
```

```
        screen.setTextColor(SSD1306_WHITE);
```

```
        screen.setCursor(screen.height()/4,  
screen.height()/8+screen.height()/2);
```

```
        screen.println("PREVIOUS");
```

```
        screen.display();
```

```
    if(digitalRead(START) == LOW) {
```

```
        screen.fillRoundRect(0, 0, screen.width(), screen.height()/2,
```

```

    screen.height()/4, SSD1306_INVERSE);
    screen.display();
    delay(200);
    return;
} else if (digitalRead(PREV) == LOW) {
    screen.fillRect(0, screen.height()/2, screen.width(),
screen.height()/2,
    screen.height()/4, SSD1306_INVERSE);
    screen.display();
    delay(200);
    displayResult();
}

}

}

```

```

void setup() {
    Serial.begin(115200);

    Serial.print("Initializing Pulse Oximeter..");

    pinMode(START, INPUT_PULLUP);
    pinMode(PREV, INPUT_PULLUP);
    pinMode(RESET, INPUT_PULLUP);

    if(!sensor.begin()) {
        Serial.println("FAILED");
    }
}

```

```
    for(;;);  
  } else {  
    Serial.println("SUCCESS");  
  }
```

```
  if(!screen.begin()) {  
    Serial.println(F("SSD_1306 Allocation Failed"));  
    for(;;);  
  }
```

```
  sensor.setOnBeatDetectedCallback(onBeatDetected);  
  
}
```

```
float lastReadHR = 0;  
float lastReadBO = 0;
```

```
float prevReadHR = 0.0;  
float PrevReadBO = 0.0;
```

```
void displayResult() {  
  while(true) {  
    screen.clearDisplay();  
    screen.setTextSize(1);  
    screen.setTextColor(SSD1306_WHITE);
```



```
    screen.setCursor(0,0);
    screen.print("HEARTRATE: ");
    screen.print(lastReadHR/30);
    screen.println(" bpm");
    screen.print("BLOOD OXYGEN: ");
    screen.print(lastReadBO/30);
    screen.println(" %");

    screen.drawRoundRect(0, screen.height()/2, screen.width(),
screen.height()/2,
    screen.height()/4, SSD1306_WHITE);
    screen.setTextSize(1);
    screen.setTextColor(SSD1306_WHITE);
    screen.setCursor(screen.height()/4,
screen.height()/8+screen.height()/2);
    screen.println("CONTINUE");
    screen.display();

    if(digitalRead(START) == LOW) {
        screen.fillRoundRect(0, screen.height()/2, screen.width(),
screen.height()/2,
        screen.height()/4, SSD1306_INVERSE);
        screen.display();
        delay(200);
        return;
    }
}
}
```

```
void loop() {  
    sensor.update();  
    lastReadBO = 0.0;  
    lastReadHR = 0.0;  
    screen.clearDisplay();  
    screen.setTextSize(1);  
    screen.setTextColor(SSD1306_WHITE);  
    screen.setCursor(0,screen.height()/2);  
    screen.println("Taking reading please wait..");  
    screen.display();  
    screen.display();  
  
    if(millis() - tsLastReport > REPORTING_PERIOD_MS) {  
        float HR = sensor.getHeartRate();  
        float BO = sensor.getSpO2();  
        screen.clearDisplay();  
        screen.display();  
        Serial.print("HR: ");  
        Serial.print(HR);  
        Serial.print("bpm\nSpO2: ");  
        Serial.print(BO);  
        Serial.println("%");  
  
        if (HR > 60 && HR < 100 && BO > 95){  
            count++;  
        }  
    }  
}
```

```
    lastReadHR += HR;
    lastReadBO += BO;
}
// Serial.println(millis());
// Serial.println(tsLastReport);

tsLastReport = millis();
}

if(digitalRead(RESET) == LOW) {
    displayMenu();
    sensor.begin();
}

if(count >= 30) {
    displayResult();
    displayMenu();
    sensor.begin();
    count = 0;
}

}
```